

03

Advanced

Distributed Services and Production Operations

DURATION

20 weeks

WEEKLY EFFORT

5-10 hours per week

CAPSTONE

**Payment System and C++
HTTP Server**

Purpose and expectations

Design and operate services that communicate through messages and RPC, expose measurable reliability signals, and scale under load. This level adds production operations and a systems-programming perspective to application engineering.

RECOMMENDED DURATION**20 weeks****WEEKLY COMMITMENT****5-10 hours per week****ENTRY CRITERIA****Level 2 and experience operating multi-service applications****PRIMARY PROJECT****Payment System and C++ HTTP Server****CORE FOCUS****Messaging, microservices, gRPC, observability, Kubernetes, performance engineering, and systems programming****How to use this roadmap**

1. Study for understanding, then validate the idea through implementation.
2. Keep evidence: source code, tests, measurements, diagrams, and short decision records.
3. Prefer a working, explained trade-off over a large but unfinished feature set.
4. Complete each assessment from a clean checkout before advancing.

Capabilities at completion

- Configure RabbitMQ exchanges, acknowledgments, dead-letter queues, publisher confirms, and prefetch.
- Design event-driven workflows with choreography, orchestration, compensation, and eventual consistency.
- Build microservices behind an API gateway and choose synchronous or asynchronous communication intentionally.
- Define unary and streaming gRPC interfaces with Protocol Buffers and benchmark them against REST.
- Design GraphQL schemas and eliminate N+1 queries with batching and DataLoader.
- Instrument services with structured logs, Prometheus metrics, Grafana dashboards, and OpenTelemetry traces.
- Deploy with Kubernetes Services, Deployments, probes, resource requests, HPA, rolling updates, and rollback.
- Profile PostgreSQL, manage connection pools, and validate performance with k6 load tests.
- Build a multithreaded C++ HTTP server with POSIX sockets and a bounded thread pool.
- Apply circuit breakers, streaming I/O, and backpressure to protect constrained dependencies.

Roadmap at a glance

PHASE	SCHEDULE	FOCUS	EVIDENCE
Messaging and RPC	Weeks 1-4	RabbitMQ, sagas, gRPC, gateways	Distributed workflow prototype
Payment capstone	Weeks 5-8	Services, resilience, observability, K8s	Payment System
Data and operations	Weeks 9-12	GraphQL, K8s, PostgreSQL, k6	Measured production deployment
Systems programming	Weeks 13-16	C++ server, documentation, assessment	Benchmark report
Directed study	Weeks 17-20	Tracing, centralized logs, hardening	Portfolio release

Completion rule

Do not treat elapsed time as completion. Advance when you can demonstrate the deliverables, explain the design decisions, and reproduce the result.

Weekly learning plan

WEEK 01

RabbitMQ and Asynchronous Messaging

3h study + 5h practice

LEARNING FOCUS

- Direct, topic, and fanout exchanges.
- Durable queues, consumer acknowledgments, publisher confirms, prefetch, and retry boundaries.
- Dead-letter exchanges and poison-message handling.

APPLIED WORK

- Create order, notification, and log exchanges.
- Bind three consumers and configure two dead-letter queues.
- Simulate a processing failure and inspect the resulting dead letter.

CONSUMER ACKNOWLEDGMENT

```
channel.consume(queue, async (message) => {
  if (!message) return;
  try {
    await processPayment(JSON.parse(message.content.toString()));
    channel.ack(message);
  } catch (error) {
    channel.nack(message, false, false); // route to the configured DLX
  }
});
```

DELIVERABLE A reproducible broker topology with failure evidence, message contracts, and operational notes.

WEEK 02

Event-Driven Architecture and Sagas

4h study + 5h practice

LEARNING FOCUS

- Event sourcing vs event-driven integration.
- Choreographed and orchestrated sagas.
- Compensating actions, idempotent consumers, and eventual consistency.

APPLIED WORK

- Model Order, Payment, and Inventory services.
- Publish success and failure events.
- Compensate a failed inventory reservation with a refund and order cancellation.
- Trace events with correlation and causation IDs.

DELIVERABLE A three-service saga that demonstrates both the happy path and a verified compensation path.

WEEK 03

gRPC and Protocol Buffers

3h study + 5h practice

LEARNING FOCUS

- Service contracts, field numbering, compatibility, and generated clients.
- Unary, server-streaming, client-streaming, and bidirectional RPC.

APPLIED WORK

- Define fraud-analysis and alert-streaming RPCs.
- Implement a Node.js server and client.
- Benchmark gRPC and REST payload, throughput, and latency with k6 or an equivalent driver.

- Deadlines, cancellation, metadata, and status handling.

PROTOCOL BUFFER CONTRACT

```
service FraudDetection {  
  rpc Analyze(Transaction) returns (FraudResult);  
  rpc StreamAlerts(AlertFilter) returns (stream Alert);  
}  
  
message FraudResult {  
  bool approved = 1;  
  double risk_score = 2;  
  string reason = 3;  
}
```

DELIVERABLE A versioned protobuf contract, working unary and streaming calls, and a concise REST comparison report.

WEEK 04

Microservices, API Gateway, and Service Discovery

3h study + 5h practice

LEARNING FOCUS

- Gateway routing, centralized cross-cutting policy, and the risks of gateway business logic.
- Client-side vs server-side discovery.
- Synchronous and asynchronous integration trade-offs.

APPLIED WORK

- Create User, Order, and Payment services.
- Route through one gateway with authentication.
- Apply role checks and per-service rate limits.
- Test discovery, routing, and degraded-service behavior.

DELIVERABLE Three small services behind an authenticated gateway with documented communication choices.

WEEKS 05-08

Capstone: Payment System

Build a resilient payment workflow with mixed languages, messaging, RPC, observability, and orchestration infrastructure.

LEARNING FOCUS

- Payments over REST and internal gRPC.
- RabbitMQ events, dead-letter handling, and saga compensation.
- Circuit breaker states and fallbacks.
- Prometheus metrics, Grafana dashboards, and Kubernetes deployment.

APPLIED WORK

- Implement a Node.js/TypeScript Payment Service and gateway.
- Implement Python Fraud Detection with gRPC and messaging.
- Persist payment state in PostgreSQL.
- Add custom throughput, latency, error, and fraud metrics.
- Deploy services with probes, resource requests, and local Compose support.

ARCHITECTURE AND SCOPE

- API Gateway: authentication and routing
- Payment Service: REST, gRPC client, events, and transactions
- Fraud Detection: Python gRPC server and event consumer
- RabbitMQ: exchanges and dead-letter paths
- PostgreSQL: payment state
- Prometheus and Grafana: service-level signals

DELIVERABLE A complete Payment System with three services, gateway, RabbitMQ and DLQ, gRPC, compensation, circuit breaker, observability, Kubernetes manifests, and local Compose.

WEEK 09

GraphQL and DataLoader

3h study + 5h practice

LEARNING FOCUS

- Schema design, nullability, resolvers, mutations, and subscriptions.
- N+1 query behavior and request-scoped batching.
- Authorization and complexity limits at the graph boundary.

APPLIED WORK

- Migrate three E-commerce queries or mutations from REST.
- Batch reviews by product ID with DataLoader.
- Measure query count before and after batching.

DELIVERABLE A GraphQL slice with request-scoped DataLoader and evidence that N+1 behavior was removed.

WEEK 10

Kubernetes Probes, Autoscaling, and Rollback

3h study + 5h practice

LEARNING FOCUS

- Startup, liveness, and readiness semantics.

APPLIED WORK

- Deploy Payment Service on kind or minikube.

- Resource requests and limits as scheduling and autoscaling inputs.
- Horizontal Pod Autoscaler, rolling updates, rollout status, and rollback.

- Configure probes and an HPA from two to ten replicas.
- Perform a rolling update, observe it, and roll it back.

HPA TARGET

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

DELIVERABLE A verified Kubernetes deployment with probes, HPA behavior, rollout evidence, and rollback instructions.

WEEK 11 PostgreSQL Performance Engineering

3h study + 5h practice

LEARNING FOCUS

- EXPLAIN (ANALYZE, BUFFERS), partial and compound indexes, and pg_stat_statements.
- Vacuum, analyze, table statistics, and concurrent index creation.
- Application pools and external pooling with PgBouncer.

APPLIED WORK

- Select three real queries from the capstones.
- Capture baseline plans and latency.
- Tune queries or indexes and repeat under the same conditions.
- Set an explicit connection budget.

DELIVERABLE A before-and-after report for three queries, including plans, latency, buffers, and the reason for each change.

WEEK 12 Load Testing with k6

2h study + 5h practice

LEARNING FOCUS

- Smoke, load, stress, spike, and soak tests.
- Virtual users, arrival rates, realistic data, and coordinated omission.
- Thresholds for p95/p99 latency and error rate.
- Backpressure, rate limits, body limits, and connection-pool saturation.

APPLIED WORK

- Test the Payment Service at 50, 100, and 200 virtual users.
- Define thresholds before running the test.
- Correlate the bottleneck with database, application, and broker metrics.

DELIVERABLE A repeatable load-test suite and a report with charts, bottleneck evidence, and the next optimization hypothesis.

WEEKS 13-14

Mini Project: C++ HTTP Server

Use systems programming to understand sockets, concurrency, memory, and the runtime cost behind application servers.

LEARNING FOCUS

- POSIX sockets, HTTP/1.1 request parsing, and response framing.
- Bounded thread pools, task queues, mutexes, and condition variables.
- Benchmarking methodology and safe resource cleanup.

APPLIED WORK

- Build a functional single-thread server, then add a configurable thread pool.
- Support basic GET and POST requests.
- Benchmark against Node.js under the same workload.
- Track throughput, p95 latency, and error rate.

DELIVERABLE A C++17 repository with `CMakeLists.txt`, working HTTP server, bounded thread pool, benchmark script, results, and limitations.

WEEK 15

Documentation and Review

LEARNING FOCUS

- Architecture documentation, decision records, runbooks, and high-signal READMEs.
- Interview practice for trees and graphs.

APPLIED WORK

- Create `ARCHITECTURE.md` for each capstone.
- Document setup, operations, failure modes, and benchmarks.
- Solve five medium tree or graph problems.

DELIVERABLE Professional project documentation that lets another engineer build, run, inspect, and discuss the system.

WEEK 16

Level Assessment

LEARNING FOCUS

- Compare RabbitMQ exchange types and saga styles.
- Explain sync vs async communication, protobuf trade-offs, and Kubernetes probe behavior.

APPLIED WORK

- Answer at least five of seven questions accurately.
- Demonstrate the Payment System and C++ server.
- Explain one production incident scenario using metrics and traces.

- Compare thread pools with thread-per-request and defend partial indexes.

DELIVERABLE Pass when the assessment threshold is met, the Payment System is complete, and the C++ server works with documented benchmarks.

WEEKS 17-20

Directed Study and Portfolio Release

LEARNING FOCUS

- OpenTelemetry tracing and context propagation.
- Centralized logging with Loki and Promtail.
- Health checks and graceful shutdown in systems code.
- System design interview communication.

APPLIED WORK

- Week 17: add distributed traces to Payment Service.
- Week 18: centralize and query logs.
- Week 19: harden the C++ server lifecycle.
- Week 20: complete a cross-project review and mock interview.

DELIVERABLE A tagged portfolio release with trace screenshots, logging queries, operational documentation, and a final retrospective.

Recommended source material

Messaging and distributed workflows

RabbitMQ tutorials and reliability guides; Martin Fowler on Event Sourcing; saga pattern guidance.

RPC and APIs

gRPC core concepts; Protocol Buffers language guide; Apollo Server and DataLoader documentation.

Operations

Kubernetes probes and autoscaling documentation; Prometheus, Grafana, and OpenTelemetry guidance.

Performance and systems

PostgreSQL performance documentation; k6 test guidance; POSIX socket and C++ concurrency references.

Completion perspective

Level 3 is complete when you can prove reliability and performance with operational evidence, not only with architecture diagrams.